

Árboles en Java

Vamos a generar un resumen conceptual para alumnos de árbol general, árbol binario, árbol BST (incluyendo recorridos LIFO y FIFO, es decir DFS y BFS) y árboles balanceados (AVL y RED-BLACK). Asimismo introducimos la idea de la interfaz SET y su implementación en un TreeSet. Luego vamos a generar un detalle de la actividad hecha donde organizabamos archivos en un árbol BST casero y luego hicimos el mismo ejemplo con TreeSet. Y finalmente la explicación de la actividad hecha de árbol general que simula el explorador de Windows. Dejamos lo demás para la clase que viene (implementación de un árbol que lee realmente la estructura de carpetas y buscar archivos y los lee).

Unidad: Árboles en Java – Resumen Conceptual y Actividades Realizadas

1. ¿Qué es un Árbol?

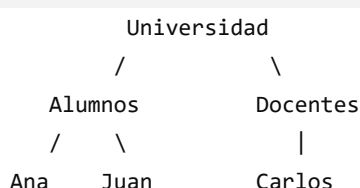
Un **árbol** es una estructura de datos jerárquica formada por nodos.

Cada nodo puede contener información y referencias a otros nodos.

Conceptos básicos:

- **Raíz (Root):** primer nodo del árbol.
- **Nodo Padre:** nodo que contiene referencias a otros nodos.
- **Nodo Hijo:** nodo dependiente de otro.
- **Hoja:** nodo sin hijos.
- **Nivel:** profundidad dentro del árbol.
- **Subárbol:** árbol contenido dentro de otro árbol.

Ejemplo:



2. Árbol General

Un **árbol general** es aquel donde un nodo puede tener cualquier cantidad de hijos.

Ejemplo:

```
C:
├── Documentos
│   ├── Trabajo.docx
│   └── TP.pdf
├── Imágenes
│   ├── Foto1.jpg
│   ├── Foto2.jpg
│   └── Foto3.jpg
└── Música
```

No existe límite de hijos por nodo.

Se utiliza para modelar:

- Sistemas de archivos.
- Organigramas.
- Árboles genealógicos.
- Menús de aplicaciones.

3. Árbol Binario

Un **árbol binario** es un caso particular donde cada nodo puede tener como máximo dos hijos:

- Hijo izquierdo.
- Hijo derecho.

Ejemplo:

```
      A
     / \
    B   C
   / \
  D   E
```

No existe ninguna regla de orden.

Solamente se limita a tener dos ramas por nodo.

4. Árbol Binario de Búsqueda (BST)

BST significa:

Binary Search Tree

Es un árbol binario que agrega una regla fundamental:

Izquierda < Nodo < Derecha

Ejemplo:

Insertando:

50, 30, 70, 20, 40, 60, 80

obtenemos:

```
      50
     /  \
    30   70
   / \  / \
  20 40 60 80
```

Ventajas

Permite:

- Insertar rápidamente.
- Buscar rápidamente.
- Mantener los datos ordenados.

5. Recorridos de un BST

Los recorridos permiten visitar todos los nodos del árbol.

DFS (Depth First Search)

DFS significa:

Búsqueda en Profundidad

Explora primero una rama completa y luego vuelve atrás.

Se implementa naturalmente mediante:

- Recursividad
- Pila (Stack)

Por eso suele asociarse con comportamiento:

LIFO
Last In First Out

PreOrden

Nodo
Izquierda
Derecha

Ejemplo:

```
  50
 /  \
30    70
```

Resultado:

50 30 70

InOrden

Izquierda
Nodo
Derecha

Resultado:

30 50 70

Particularidad:

En un BST devuelve los elementos ordenados.

PostOrden

Izquierda
Derecha
Nodo

Resultado:

30 70 50

6. BFS (Breadth First Search)

BFS significa:

Búsqueda por Amplitud

Recorre nivel por nivel.

Ejemplo:

```

      50
     /  \
    30   70
   / \  / \
  20 40 60 80
  
```

Resultado:

50
30 70
20 40 60 80

O:

50 30 70 20 40 60 80

¿Cómo se implementa?

Con una cola.

FIFO
First In First Out

Por eso BFS suele asociarse a:

Java

Queue

7. Problema de los BST

Los BST pueden desbalancearse.

Ejemplo:

Insertando:

```
10
20
30
40
50
```

obtenemos:

```
10
 \
 20
  \
 30
  \
 40
  \
 50
```

Prácticamente se convierte en una lista.

La búsqueda deja de ser eficiente.

8. Árboles Balanceados

Para solucionar ese problema existen árboles auto-balanceados.

AVL

Creado por:

- Adelson-Velsky

- Landis

Características:

- Mantiene equilibradas ambas ramas.
- Diferencia de altura máxima:

```
-1
0
+1
```

Si se rompe esa condición:

- realiza rotaciones.

Ejemplo:

```
30
/
20
/
10
```

Se rota para quedar:

```
  20
 /  \
10   30
```

Ventaja:

- Búsquedas extremadamente rápidas.

Desventaja:

- Más rotaciones.

Red-Black Tree

Cada nodo posee un color:

```
Rojo
Negro
```

Reglas especiales garantizan que el árbol permanezca aproximadamente balanceado.

Ventajas:

- Menos rotaciones que AVL.
- Muy eficiente para inserciones y eliminaciones.

9. ¿Cuál utiliza Java?

La implementación interna de:

Java

TreeSet

TreeMap

utiliza un:

Red-Black Tree

Por eso:

Java

TreeSet

mantiene automáticamente:

- Ordenamiento.
- Balanceo.
- Búsquedas eficientes.

10. La Interfaz Set

La interfaz:

Java

Set<E>

representa una colección sin elementos repetidos.

Ejemplo:

Java

```
Set<String> nombres = new HashSet<>();
```

```
nombres.add("Ana");
```

```
nombres.add("Ana");
```

```
nombres.add("Juan");
```

Resultado:


```
Ana
Juan
```

No admite duplicados.

11. TreeSet

Implementación de:

```
</> Java
```

```
Set<E>
```

basada en Red-Black Tree.

```
</> Java
```

```
TreeSet<String> archivos = new TreeSet<>();
```

Características:

- ✓ No admite repetidos
- ✓ Mantiene ordenados los datos
- ✓ Balanceado automáticamente
- ✓ Búsquedas rápidas

Actividad Realizada 1

Organización de archivos usando BST Casero

Objetivo:

Representar una carpeta con varios archivos utilizando un BST implementado por nosotros.

Datos cargados

Ejemplo:

```
informe.pdf
foto.jpg
tp.docx
```

```
notas.txt  
resumen.pdf  
java.pdf  
ejercicio.docx
```

Inserción

Cada archivo se inserta comparando su nombre.

```
</> Java  
compareTo()
```

determina:

- izquierda
- derecha

siguiendo la regla:

```
Izquierda < Nodo < Derecha
```

Recorridos realizados

DFS

Mediante:

- PreOrden
- InOrden
- PostOrden

BFS

Mediante:

```
</> Java  
Queue<Nodo>
```

recorriendo nivel por nivel.

Objetivo pedagógico

Comprender:

- estructura interna de un BST
 - recursividad
 - nodos enlazados
 - DFS
 - BFS
-

Actividad Realizada 2

Organización de archivos usando TreeSet

Luego se resolvió el mismo problema utilizando:

```
</> Java  
TreeSet<String>
```

Ejemplo:

```
</> Java  
TreeSet<String> archivos = new TreeSet<>();  
  
archivos.add("informe.pdf");  
archivos.add("foto.jpg");  
archivos.add("tp.docx");
```

Resultado

Automáticamente:

- quedan ordenados.
 - no hay duplicados.
 - no debemos programar nodos.
 - no debemos programar inserciones.
 - no debemos programar balanceo.
-

Conclusión

Nuestro BST casero sirve para comprender cómo funcionan los árboles.

TreeSet es la solución profesional que Java ofrece para resolver el mismo problema de forma eficiente.

Actividad Realizada 3

Árbol General simulando el Explorador de Windows

Objetivo:

Representar la estructura de carpetas de Windows mediante un árbol general.

Ejemplo:

```
C:
├── Documentos
│   ├── TP1
│   ├── TP2
│   └── Parciales
├── Imágenes
│   ├── Vacaciones
│   └── Trabajo
└── Música
```

Implementación

Cada nodo representa:

- una carpeta
- o un archivo

y contiene una lista de hijos:

```
</> Java
ArrayList<NodoGeneral>
```

Operaciones realizadas

Crear carpetas

```
</> Java
Documentos
Imágenes
Música
```

Agregar subcarpetas

</> Java

TP1
TP2
Vacaciones
Trabajo

Mostrar estructura

Salida:

```
C:
├─ Documentos
│   ├─ TP1
│   └─ TP2
├─ Imágenes
│   ├─ Vacaciones
│   └─ Trabajo
└─ Música
```

Objetivo pedagógico

Comprender cómo los sistemas operativos organizan la información.

El Explorador de Windows, Linux y macOS utilizan internamente estructuras jerárquicas equivalentes a árboles generales.

Próxima Clase

En la siguiente etapa abandonaremos los datos cargados manualmente y construiremos un árbol utilizando la estructura real del sistema de archivos mediante:

</> Java

File
FileReader
BufferedReader

Trabajaremos con:

- Lectura real de carpetas.
- Construcción automática del árbol.
- Búsqueda de archivos.
- Lectura del contenido de archivos.

- Recorridos sobre estructuras reales del sistema operativo.



Excelente observación. De hecho, antes de introducir **TreeSet** conviene explicar **Comparable**, porque es justamente el mecanismo que permite ordenar objetos dentro de un BST, un AVL, un Red-Black Tree o un TreeSet.

Podrías agregar el siguiente apartado entre **TreeSet** y las actividades prácticas.

12. Comparación de Objetos en Árboles

La Interfaz Comparable

Hasta ahora trabajamos con:

```
</> Java
```

```
Integer  
String  
Double
```

Estos tipos ya saben compararse entre sí.

Por ejemplo:

```
</> Java
```

```
"Juan".compareTo("Pedro")
```

devuelve un número negativo porque:

```
Juan < Pedro
```

Mientras que:

```
</> Java
```

```
50.compareTo(20)
```

devuelve un número positivo porque:

```
50 > 20
```

¿Por qué es importante?

Los árboles ordenados necesitan saber:

```
¿Va a la izquierda?  
¿Va a la derecha?
```

Para eso deben comparar objetos.

Cuando trabajamos con nuestros propios objetos Java no sabe cómo ordenarlos.

Ejemplo:

```
</> Java  
  
public class Alumno {  
  
    private String nombre;  
    private Integer legajo;  
  
}
```

Si intentamos insertar objetos Alumno en un BST:

```
</> Java  
  
arbol.insertar(alumno);
```

Java no sabe si un alumno es mayor o menor que otro.

Debemos enseñarle.

La Interfaz Comparable

Java proporciona la interfaz:

```
</> Java  
  
Comparable<T>
```

que obliga a implementar:

```
</> Java  
  
compareTo()
```

Ejemplo

```
</> Java
```

```
public class Alumno
    implements Comparable<Alumno> {

    private String nombre;
    private Integer legajo;

    @Override
    public int compareTo(Alumno otro) {

        return this.legajo.compareTo(
            otro.legajo
        );
    }
}
```

Ahora los alumnos se ordenarán por legajo.

¿Qué devuelve compareTo()?

Menor que

Java

```
return -1;
```

o cualquier número negativo.

Objeto actual < objeto recibido

Igual

Java

```
return 0;
```

Son iguales

Mayor que

Java

```
return 1;
```

o cualquier número positivo.

Objeto actual > objeto recibido

Ejemplo Visual

Supongamos:

`</> Java`

```
Alumno a1 = new Alumno("Ana", 100);  
Alumno a2 = new Alumno("Juan", 200);
```

Al ejecutar:

`</> Java`

```
a1.compareTo(a2)
```

obtenemos:

-100

porque:

100 < 200

Entonces el árbol sabe que:

Ana va a la izquierda de Juan

Comparable aplicado a un BST

Nuestro árbol genérico fue declarado así:

`</> Java`

```
public class Arbol<T extends Comparable<T>>
```

Esto significa:

El árbol sólo acepta objetos que sepan compararse.

Por eso luego podemos hacer:

`</> Java`

```
dato.compareTo(nodo.getDato())
```

para decidir dónde insertar.

Ejemplo:

```
</> Java
if (dato.compareTo(
    nodo.getDato()) < 0) {

    nodo.setIzquierdo(...);

} else {

    nodo.setDerecho(...);

}
```

Comparable aplicado a TreeSet

TreeSet también necesita comparar elementos.

Ejemplo:

```
</> Java
TreeSet<Alumno> alumnos =
    new TreeSet<>();
```

Para funcionar correctamente:

```
</> Java
Alumno
```

debe implementar:

```
</> Java
Comparable<Alumno>
```

Ejemplo

```
</> Java
TreeSet<Alumno> alumnos =
    new TreeSet<>();
```

```
alumnos.add(  
    new Alumno("Pedro",300));  
  
alumnos.add(  
    new Alumno("Ana",100));  
  
alumnos.add(  
    new Alumno("Juan",200));
```

Salida:

```
Ana 100  
Juan 200  
Pedro 300
```

Quedan ordenados automáticamente por legajo.

¿Qué ocurre si no implementamos Comparable?

Java lanzará una excepción:

```
ClassCastException
```

o un error similar indicando que no sabe cómo comparar los objetos.

Resumen

Estructura	Necesita comparar	Utiliza Comparable
BST Casero	Sí	Sí
AVL	Sí	Sí
Red-Black	Sí	Sí
TreeSet	Sí	Sí
TreeMap	Sí	Sí
HashSet	No	No
HashMap	No	No

Idea Fundamental para Recordar

Los árboles ordenados funcionan gracias a las comparaciones.

```
HashSet → usa hashCode()  
HashMap → usa hashCode()  
  
BST → usa compareTo()  
AVL → usa compareTo()  
RedBlack → usa compareTo()  
TreeSet → usa compareTo()  
TreeMap → usa compareTo()
```

Por esa razón, cuando comenzamos a trabajar con árboles genéricos, apareció la restricción:

```
</> Java  
<T extends Comparable<T>>
```

que garantiza que cualquier objeto almacenado en el árbol pueda ser comparado y ordenado correctamente.



bueno dame un ejemplo concreto sencillo de un TreeSet que falle por no implementar comparable con el código completo

Este ejemplo es ideal para mostrar en clase **por qué existe Comparable**.

Paso 1: Clase Alumno SIN Comparable

```
</> Java  
  
package ejemplo;  
  
public class Alumno {  
  
    private String nombre;  
    private int legajo;  
  
    public Alumno(String nombre, int legajo) {  
        this.nombre = nombre;  
        this.legajo = legajo;  
    }  
  
    @Override  
    public String toString() {
```

```
        return nombre + " - " + legajo;
    }
}
```

Paso 2: Uso de TreeSet

```
</> Java
package ejemplo;

import java.util.TreeSet;

public class App {

    public static void main(String[] args) {

        TreeSet<Alumno> alumnos = new TreeSet<>();

        alumnos.add(new Alumno("Juan", 200));
        alumnos.add(new Alumno("Ana", 100));
        alumnos.add(new Alumno("Pedro", 300));

        System.out.println(alumnos);
    }
}
```

¿Qué ocurre?

Compila correctamente.

Pero al ejecutar aparece algo parecido a:

```
Exception in thread "main"
java.lang.ClassCastException:
class ejemplo.Alumno cannot be cast to class java.lang.Comparable
```

o según la versión:

```
java.lang.ClassCastException:
ejemplo.Alumno cannot be cast to java.lang.Comparable
```

¿Por qué falla?

Porque internamente TreeSet necesita decidir:

¿Ana va antes que Juan?
¿Pedro va después que Juan?

Para eso utiliza:

```
</> Java  
compareTo()
```

Pero la clase Alumno no tiene ese método.

Entonces Java no sabe cómo ordenarlos.

Solución: implementar Comparable

```
</> Java  
package ejemplo;  
  
public class Alumno  
    implements Comparable<Alumno> {  
  
    private String nombre;  
    private int legajo;  
  
    public Alumno(String nombre, int legajo) {  
        this.nombre = nombre;  
        this.legajo = legajo;  
    }  
  
    @Override  
    public int compareTo(Alumno otro) {  
        return Integer.compare(  
            this.legajo,  
            otro.legajo  
        );  
    }  
  
    @Override  
    public String toString() {  
        return nombre + " - " + legajo;  
    }  
}
```

Ahora el mismo App funciona

```
</> Java
```

```
package ejemplo;

import java.util.TreeSet;

public class App {

    public static void main(String[] args) {

        TreeSet<Alumno> alumnos = new TreeSet<>();

        alumnos.add(new Alumno("Juan", 200));
        alumnos.add(new Alumno("Ana", 100));
        alumnos.add(new Alumno("Pedro", 300));

        System.out.println(alumnos);
    }
}
```

Salida:

```
[Ana - 100, Juan - 200, Pedro - 300]
```

Lo interesante para remarcar en clase

Preguntas:

"¿Por qué ArrayList funciona sin Comparable y TreeSet no?"

Este código funciona perfectamente:

```
</> Java

ArrayList<Alumno> alumnos =
    new ArrayList<>();

alumnos.add(new Alumno("Juan", 200));
alumnos.add(new Alumno("Ana", 100));
alumnos.add(new Alumno("Pedro", 300));

System.out.println(alumnos);
```

Porque una lista simplemente guarda objetos.

No necesita ordenarlos.

En cambio:

```
</> Java
```

```
TreeSet<Alumno>
```

internamente utiliza un árbol Red-Black y necesita comparar constantemente los elementos para decidir:

```
izquierda o derecha
menor o mayor
antes o después
```

Por eso exige:

```
</> Java
implements Comparable<Alumno>
```

Esta es probablemente la demostración más sencilla para que los alumnos entiendan la diferencia entre una colección que **almacena** objetos (ArrayList) y una que además debe **ordenarlos** (TreeSet, TreeMap, BST, AVL y Red-Black).



no quiero que use compare... que use compareTo

Perfecto. Si todavía no viste métodos auxiliares como `Integer.compare()`, es mucho más didáctico usar directamente `compareTo()`, porque además es exactamente el mismo mecanismo que usa un BST genérico.

La clase quedaría así:

```
</> Java
package ejemplo;

public class Alumno
    implements Comparable<Alumno> {

    private String nombre;
    private Integer legajo;

    public Alumno(String nombre, Integer legajo) {
        this.nombre = nombre;
        this.legajo = legajo;
    }

    @Override
    public int compareTo(Alumno otro) {

        return this.legajo.compareTo(
            otro.legajo
        );
    }
}
```



```
    );  
  
    }  
  
    @Override  
    public String toString() {  
        return nombre + " - " + legajo;  
    }  
}
```

Y el App :

```
<> Java  
package ejemplo;  
  
import java.util.TreeSet;  
  
public class App {  
  
    public static void main(String[] args) {  
  
        TreeSet<Alumno> alumnos =  
            new TreeSet<>();  
  
        alumnos.add(  
            new Alumno("Juan", 200));  
  
        alumnos.add(  
            new Alumno("Ana", 100));  
  
        alumnos.add(  
            new Alumno("Pedro", 300));  
  
        for (Alumno alumno : alumnos) {  
            System.out.println(alumno);  
        }  
    }  
}
```

Salida:

```
Ana - 100  
Juan - 200  
Pedro - 300
```

Incluso podés mostrarles cómo funciona el `compareTo()` a mano:

```
<> Java
```

```
Integer a = 100;  
Integer b = 200;  
  
System.out.println(a.compareTo(b));
```

Resultado:

-1

porque:

$100 < 200$

Otro ejemplo:

```
</> Java  
System.out.println(200.compareTo(100));
```

Resultado:

1

porque:

$200 > 100$

Y:

```
</> Java  
System.out.println(100.compareTo(100));
```

Resultado:

0

porque:

$100 = 100$

Entonces el razonamiento que hace el árbol es:

```
</> Java  
if (nuevo.compareTo(actual) < 0) {  
    // izquierda  
}  
else if (nuevo.compareTo(actual) > 0) {
```

```
        // derecha  
    }  
    else {  
        // repetido  
    }  
}
```

Esa es exactamente la misma lógica que implementaste en tu BST genérico con:

🔗 Java

```
public class Arbol<T extends Comparable<T>>
```

por lo que sirve como puente perfecto entre el BST casero y el `TreeSet`.

